

Aplikacje bazodanowe – projekt egzaminacyjny

Zadanie

Zaprojektuj i zrealizuj serwis REST obsługujący „bazę rodowodową” o strukturze i funkcjonalności opisanych poniżej.

- W bazie przechowywane są dane o **krajach**, **hodowcach** oraz **koniach** czystej krwi arabskiej.
- Dla każdego **kraju** przechowujemy jego dwuliterowy kod ISO (np. **PL**, **DE**, **IT**, **US**, ...) oraz pełną nazwę w języku polskim.
- Dla każdego **hodowcy** przechowujemy jego nazwę, odwołanie do informacji o kraju w jakim prowadzi(ł) swoją hodowlę oraz (opcjonalny) atrybut o wartości tekstowej przeznaczony na „notatki”.
- Dla każdego **konia** przechowujemy
 - nazwę (imię),
 - informację o roku urodzenia,
 - płeć,
 - maść,
 - odwołanie do danych na temat kraju urodzenia (**uwaga** – może być inny niż kraj hodowcy),
 - opcjonalne odwołania do informacji o ojcu oraz matce,
 - odwołanie do danych hodowcy,
 - opcjonalne „notatki” (podobnie jak w przypadku hodowców).
- Oprócz danych „rodowodowych” (związanych z końmi) w bazie powinny znaleźć się również informacje na temat „administratorów” (zawierające co najmniej login i hasz hasła). Tylko administratorzy powinni mieć możliwość wprowadzania, modyfikowania oraz pełnego odczytywania danych.

Uwagi

- Płeć konia to: *klacz*, *ogier* bądź *wałach*
- Dla uproszczenia zakładamy, że dostępne maści koni to wyłącznie: *siwa*, *gniada*, *kasztanowata* lub *kara*.
- Dodatkowo, także dla uproszczenia, zakładamy, że:
 - napisy postaci „*imię_konia XX 9999*”, gdzie „XX” jest kodem kraju urodzenia konia, a „9999” jego rokiem urodzenia, są unikatowe w ramach bazy (**uwaga** – nie należy używać takich napisów jako „kluczy technicznych” w bazie)
 - podobnie unikatowe są napisy postaci „*nazwa_hodowcy XX*”, gdzie „XX” jest kodem kraju, w którym hodowca prowadzi(ł) swoją działalność.

Dzięki takiemu założeniu, odwołując się do konkretnego konia, możemy użyć jego imienia (jeśli jest ono unikatowe w ramach bazy), imienia z rocznikiem, a w skrajnym wypadku pełnego „identyfikatora użytkowego” składającego się z imienia, kodu kraju urodzenia oraz rocznika. Podobnie postępujemy ilekroć z poziomu REST API chcemy odwołać się do konkretnego hodowcy. – tutaj „identyfikatorem użytkowym” jest albo nazwa hodowcy wraz z kodem kraju albo sama nazwa (o ile jest unikatowa w ramach kolekcji zawierającej dane hodowców).

- Pamiętamy, że matką konia może być jedynie *klacz*, a ojcem – *ogier*
- Zakładamy (nieco arbitralnie), że klacz/ogier, aby zostać matką/ojcem, w momencie urodzenia źrebięcia, musi być w wieku od 3 do 21 lat.

Serwis REST-owy

- Dodawanie, edycja, pobieranie oraz usuwanie danych na temat krajów, hodowców, oraz koni (dostępne jedynie dla administratorów).
- Pobieranie danych na temat rodowodu konia z zadaną „głębokością”, czyli liczbą generacji (np. do pradiadków włącznie)
- Pobieranie informacji o potomstwie danego konia (z opcjonalnym „filtrem” dotyczącym płci i/lub hodowcy).

Uwaga

- Operacje REST-owe, w procesie wprowadzania oraz edycji danych **nie powinny** bezpośrednio używać „identyfikatorów technicznych” (typu `ObjectId`) dokumentów. Należy twórczo wykorzystać założenia co do unikatowości „identyfikatorów użytkowych” opisanych powyżej.

Interfejs webowy

Rozwiązanie powinno oferować

- Możliwość tworzenia „wizualizacji” rodowodu konia w formie HTML obrazującego „drzewo genealogiczne” konia (z zadaną liczbą generacji).

- Opcjonalnie (na ocenę > 3.5) interfejs obsługujący operacje „administracyjne” (dodawanie, modyfikowanie, przeglądanie oraz usuwanie danych).

Wymagania techniczne wobec rozwiązania

- W realizacji projektu należy wykorzystać język `JavaScript`, a także narzędzia: `Node.js`, `Faker.js`, `Express.js` oraz `passport.js`.
- Serwis REST powinien brać pod uwagę potencjalnie błędne dane i dbać o to, aby w bazie zapisywane były jedynie dane „formalnie poprawne”.
- W rozwiązaniu należy wykorzystać bazę `MongoDB` oraz bibliotekę `Mongoose` (lub opcjonalnie sterownik `MongoDB` z użyciem „schematów `MongoDB`”). Serwer baz danych powinien działać w środowisku `Docker`.

Egzamin – założenia/wytyczne

- Na egzamin przychodzimy z laptopem, na którym zainstalowana jest aplikacja, a baza wypełniona jest co najmniej 6 pokoleniami danych (z wykorzystaniem skryptów bazujących na narzędziu `Faker.js`).
- Konfiguracja laptopa powinna umożliwiać dostęp sieciowy do serwisu REST-owego.